

Where irrigation exists is globally contested

1. Original datasets: from raster to tabular format

Arnald Puy

Contents

1	Preliminary	2
2	Global irrigated area maps	2
2.1	Meier et al map	2
2.2	GIAM	5
2.3	GMIA v5.0 (Siebert et al.)	7
2.4	Nagaraj et al.	9
2.5	MIRCA 2000	11
2.6	GAEZ+2015	12
2.7	SPAM2010	14
2.8	GRIPC	16
2.9	MIRCA-OS	20
2.10	LUH2	21
3	Export	24
4	Session information	25

1 Preliminary

```
# PRELIMINARY FUNCTIONS #####
#####

sensobol::load_packages(c("data.table", "terra", "here", "countrycode",
                          "rworldmap", "sp", "sf", "rnaturalearth", "readxl",
                          "tidyverse", "scales", "ncdf4"))

# Source all helper functions -----

r_functions <- list.files(path = here("functions"), pattern = "\\R$",
                          full.names = TRUE)

invisible(lapply(r_functions, source))

# Reproducibility -----

set.seed(123L)

# Countries layer (WGS84, only name + geometry) -----

countries_sf <- rnaturalearth::ne_countries(scale = "medium",
                                             returnclass = "sf") %>%
  .[, c("name", "geometry")]

# Output folder for all original data tables -----

out_dir <- here("original_datasets")
if (!dir.exists(out_dir)) dir.create(out_dir, recursive = TRUE)

# Columns to keep for gridded products -----

cols_to_retrieve <- c("lon", "lat", "country", "continent", "mha", "dataset")
```

2 Global irrigated area maps

2.1 Meier et al map

```
# MEIER ET AL. #####
# - 1 km grid; keep class 1 pixels and convert to Mha per cell

f_meier <- "./irrigated_area_datasets/meier_et_al/global_irrigated_areas.tif"
r_meier <- rast(f_meier)

meier_dt <- as.data.table(as.data.frame(r_meier, xy = TRUE, na.rm = TRUE))
setnames(meier_dt, old = c("x", "y", "global_irrigated_areas"),
```

```

new = c("lon", "lat", "class"))

# Keep irrigated areas -----
# LEGEND:
# 0 = no irrigated area",
# 1 = downscaled Siebert et al. 2013",
# 2 = low suitability + high NDVI + NDVI vegetation course",
# 3 = potential multiple cropping < actual multiple cropping",
# 4 = cropland (ESA-CCI-LC/GlobCover) + low suitability",

meier_dt <- meier_dt[class %in% 1:4]

# Pixel area at 30 arc-sec (1/120 deg) -----

dlat <- 1 / 120
dlon <- 1 / 120
R <- 6371008.8 # m

meier_dt[, cell_area_m2:= (R^2) * (dlon * pi / 180) * (sin((lat + dlat / 2) * pi / 180) -
sin((lat - dlat / 2) * pi / 180))]
meier_dt[, cell_area_m2:= abs(cell_area_m2)]
meier_dt[, cell_area_ha:= cell_area_m2 / 1e4]
meier_dt[, cell_area_mha:= cell_area_ha / 1e6]

# Countries + continents -----

meier_dt <- add_country_continent(meier_dt, countries_sf, chunk_size = 500000L)

## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, Somaliland, Taiwan
## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan

# Dataset + mha column -----

meier_dt[, dataset:= "meier"]
meier_dt[, mha:= cell_area_mha]

meier_dt <- meier_dt[, ..cols_to_retrieve]
meier_dt <- na.omit(meier_dt)

# Check totals-----

# Original paper reports 367 Mha (p.1124)
meier_dt[, sum(mha)]

## [1] 366.289

# Export-----

```

```
fwrite(meier_dt, file.path(out_dir, "meier_dt.csv"))
```

2.2 GIAM

```
# GIAM (Thenkabail et al. 2009) #####
# - 30 arc-sec grid; class-based IAFs to TAAI + AIA, we use TAAI Mha

f_giam <- "./irrigated_area_datasets/giam/giam_28_classes_global.tif"
r_giam <- rast(f_giam)

giam_dt <- as.data.table(as.data.frame(r_giam, xy = TRUE, na.rm = TRUE))

# Drop non-irrigated class 0 and ensure integer class codes -----

giam_dt <- giam_dt[giam_28_classes_global != 0]
giam_dt[, giam_28_classes_global := as.integer(giam_28_classes_global)]
setnames(giam_dt, c("x", "y"), c("lon", "lat"))

# Pixel area (30 arc-sec) -----

dlat <- 1 / 120
dlon <- 1 / 120
R <- 6371008.8

giam_dt[, cell_area_m2 := abs((R^2) * (dlon * pi / 180) *
                             (sin((lat + dlat / 2) * pi / 180) -
                              sin((lat - dlat / 2) * pi / 180))))]
giam_dt[, cell_area_ha := cell_area_m2 / 1e4]
giam_dt[, cell_area_mha := cell_area_m2 / 1e10]

# IAF table (class → seasonal fractions) -----

iaf_xlsx <- "./irrigated_area_datasets/giam/GIAM_IAF_Table.xlsx"
iaf <- as.data.table(readxl::read_excel(iaf_xlsx, sheet = "GIAM_IAF_Table"))
setnames(iaf, tolower(names(iaf)))

iaf[, s1_mean := rowMeans(.SD, na.rm = TRUE), .SDcols = c("s1_hri", "s1_spdt")]
iaf[, s2_mean := rowMeans(.SD, na.rm = TRUE), .SDcols = c("s2_hri", "s2_spdt")]
iaf[, cont_mean := rowMeans(.SD, na.rm = TRUE), .SDcols = c("cont_hri", "cont_spdt")]

for (cc in c("s1_mean", "s2_mean", "cont_mean")) {
  iaf[is.na(get(cc)), (cc) := NA_real_]
}

setkey(iaf, class)
setkey(giam_dt, giam_28_classes_global)

giam_dt[iaf, `:=`(iaf_gee = i.iaf_gee, iaf_s1 = i.s1_mean,
                  iaf_s2 = i.s2_mean, iaf_cont = i.cont_mean)]
```

```

# Net irrigated area (TAAI) + annualized area (AIA) -----

giam_dt[, irrig_taa_mha:= cell_area_mha * iaf_gee]

giam_dt[, irrig_aia_mha:= cell_area_mha * (fcoalesce(iaf_s1, 0) + fcoalesce(iaf_s2, 0) +
                                              fcoalesce(iaf_cont, 0))]
giam_dt[, irrig_taa_ha:= irrig_taa_mha * 1e6]
giam_dt[, irrig_aia_ha:= irrig_aia_mha * 1e6]

# Countries + continents -----

giam_dt <- add_country_continent(giam_dt, countries_sf, chunk_size = 500000L)

## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, Somaliland, Taiwan, U
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, U.S. Virgin Is.
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:

# Dataset + mha (use TAAI) -----

giam_dt <- giam_dt[!is.na(country) & !is.na(continent)]
giam_dt[, dataset:= "giam"]
giam_dt[, mha:= irrig_taa_mha]

giam_dt <- giam_dt[, ..cols_to_retrieve]

# Check totals-----

# Original paper reports 398 Mha (Table 3)
giam_dt[, sum(mha)]

## [1] 343.1581

# Export-----

fwrite(giam_dt, file.path(out_dir, "giam_dt.csv"))

```

2.3 GMIA v5.0 (Siebert et al.)

```
# GMIA v5.0 (Siebert et al.) #####
# - 5 arc-min grid; AEI × AAI% = actually irrigated Mha per cell

r_gmia_aei <- rast("./irrigated_area_datasets/gmia/gmia_v5_aei_ha.asc")
gmia_dt <- as.data.table(as.data.frame(r_gmia_aei, xy = TRUE, na.rm = FALSE))

r_gmia_pct <- rast("./irrigated_area_datasets/gmia/gmia_v5_aai_pct_aei.asc")
gmia_dt_pct <- as.data.table(as.data.frame(r_gmia_pct, xy = TRUE, na.rm = FALSE))

setkey(gmia_dt, x, y)
setkey(gmia_dt_pct, x, y)
gmia_dt <- gmia_dt[gmia_dt_pct]

# Remove non-irrigated cells and convert percentage to fraction -----

gmia_dt[, gmia_v5_aai_pct_aei:= gmia_v5_aai_pct_aei / 100]
gmia_dt[, gmia_v5_aei_mha:= gmia_v5_aei_ha / 1e6]

setnames(gmia_dt, c("x", "y"), c("lon", "lat"))

# Countries + continents -----

gmia_dt <- add_country_continent(gmia_dt, countries_sf, chunk_size = 200000L)

## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
gmia_dt <- na.omit(gmia_dt)

# Dataset + mha (AEI × AAI fraction) -----

gmia_dt[, dataset:= "gmia"]
gmia_dt[, mha:= gmia_v5_aei_mha * gmia_v5_aai_pct_aei]

gmia_dt <- gmia_dt[, ..cols_to_retrieve]

# Check totals-----

# Original paper reports 255 Mha (p. 14)
gmia_dt[, sum(mha)]

## [1] 253.0108
```

```
# Export-----  
fwrite(gmia_dt, file.path(out_dir, "gmia_dt.csv"))
```


2.4 Nagaraj et al.

```
# NAGARAJ ET AL. (2021) #####
# - 5 arc-min; classes → representative fractions × fixed 8604 ha per pixel

r_nag <- rast("./irrigated_area_datasets/nagaraj_et_al/v3b_combined_2015.tif")
nagaraj_dt <- as.data.table(as.data.frame(r_nag, xy = TRUE, na.rm = FALSE))

# Keep irrigation classes only -----
nagaraj_dt <- nagaraj_dt[classification %in% c(1, 2)]

pixel_area_ha <- 8604 # fixed by paper

# Irrigation classes represent the fraction of a 5-arc-minute pixel that is
# irrigated. Pixels classified as low-medium irrigation correspond to approximately
# 1-20% of the pixel being irrigated. Pixels classified as high-intensity irrigation
# correspond to more than 20% of the pixel being irrigated.
# To estimate irrigated area, we assign representative irrigated fractions
# of 10% to the low-medium irrigation class and 35% to the high-intensity class.
# Irrigated area is then computed by multiplying these fractions by the
# nominal 5-arc-minute pixel area (8604 ha).
# This procedure is consistent with the class definitions and training
# thresholds described in Nagaraj et al. (2021).

nagaraj_dt[, irrig_frac:= fifelse(
  classification == 1, 0.10, # low-medium: 1-20%
  fifelse(classification == 2, 0.35, NA_real_) # high: >20%
)]

nagaraj_dt[, irrig_area_ha:= irrig_frac * pixel_area_ha]
nagaraj_dt[, irrig_area_mha:= irrig_area_ha / 1e6]

setnames(nagaraj_dt, c("x", "y"), c("lon", "lat"))

# Countries + continents -----

nagaraj_dt <- add_country_continent(nagaraj_dt, countries_sf, chunk_size = 200000L)

## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, St. Vin. and Gren., 
## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, St. Vin. and Gren.

nagaraj_dt[, dataset:= "nagaraj"]
nagaraj_dt[, mha:= irrig_area_mha]
nagaraj_dt <- na.omit(nagaraj_dt)

nagaraj_dt <- nagaraj_dt[, ..cols_to_retrieve]

# Check totals-----
```

```
# Original paper does not report a total  
nagaraj_dt[, sum(mha)]  
  
## [1] 297.2691  
  
# Export-----  
  
fwrite(nagaraj_dt, file.path(out_dir, "nagaraj_dt.csv"))
```

2.5 MIRCA 2000

```
# MIRCA 2000 #####
# - Maximum area equipped for irrigation per cell (ha to Mha)

r_mirca <- rast("./irrigated_area_datasets/MIRCA_2000/MAX_CROPPED_AREA_IRC_HA.ASC")
mirca_dt <- as.data.table(as.data.frame(r_mirca, xy = TRUE, na.rm = FALSE))

mirca_dt <- mirca_dt[MAX_CROPPED_AREA_IRC_HA != 0]
mirca_dt[, mha := MAX_CROPPED_AREA_IRC_HA / 1e6]

setnames(mirca_dt, c("x", "y"), c("lon", "lat"))

# Countries + continents -----

mirca_dt <- add_country_continent(mirca_dt, countries_sf, chunk_size = 500000L)

## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, Somaliland, Taiwan, U
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, U.S. Virgin Is.
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:

mirca_dt[, dataset := "mirca2000"]
mirca_dt <- mirca_dt[, ..cols_to_retrieve]
mirca_dt <- mirca_dt[!is.na(country)]

# Check totals-----

# Original paper does not report a total
mirca_dt[, sum(mha)]

## [1] 216.0765

# Export-----

fwrite(mirca_dt, file.path(out_dir, "mirca_dt.csv"))
```

2.6 GAEZ+2015

```
# GAEZ+ 2015 #####
# - Summed irrigated harvested area per cell, capped by physical cell area

gaze_files <- list.files("./irrigated_area_datasets/GAEZ+_2015",
                        pattern = "\\..tif$", full.names = TRUE)
r_gaze <- rast(gaze_files)
gaze_dt <- as.data.table(as.data.frame(r_gaze, xy = TRUE, na.rm = FALSE))

crop_cols <- grep("^GAEZAct2015_HarvArea_.*_Irrigated$",
                 names(gaze_dt), value = TRUE)

# Total irrigated harvested area per cell (1000 ha) -----

gaze_dt[, irrig_iharv_1000ha := rowSums(.SD, na.rm = TRUE), .SDcols = crop_cols]

gaze_dt[, `:=`(irrig_iharv_ha = irrig_iharv_1000ha * 1000,
              irrig_iharv_mha = irrig_iharv_1000ha / 1000)]

# Physical cell area (approx, 0.0833° grid) -----

dlat <- 1 / 12
dlon <- 1 / 12
R <- 6371008.8

gaze_dt[, cell_area_ha := abs((R^2) * (dlon * pi / 180) *
                             (sin((y + dlat / 2) * pi / 180) -
                              sin((y - dlat / 2) * pi / 180))) / 1e4]

# Cap irrigated area by physical extent -----

gaze_dt[, irrig_extent_ha := pmin(cell_area_ha, irrig_iharv_ha)]
gaze_dt[, irrig_extent_mha := irrig_extent_ha / 1e6]

gaze_dt <- gaze_dt[!is.na(irrig_iharv_1000ha)]
setnames(gaze_dt, c("x", "y"), c("lon", "lat"))

# Countries + continents -----

gaze_dt <- add_country_continent(gaze_dt, countries_sf, chunk_size = 200000L)
```

```
## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
```

```

gaez_dt <- gaez_dt[!is.na(country)]

gaez_dt[, dataset:= "gaez_v4"]
gaez_dt[, mha:= irrig_extent_mha]

gaez_dt <- gaez_dt[, ..cols_to_retrieve]

# Check totals-----

# Original paper does not report a total
gaez_dt[, sum(mha)]

## [1] 287.8335

# Export-----

fwrite(gaez_dt, file.path(out_dir, "gaez_dt.csv"))

```

2.7 SPAM2010

```
# SPAM 2010 #####
# - Sum irrigated crop areas per cell, capped by cell area (Mha)

spam_files <- list.files("./irrigated_area_datasets/SPAM_2010", pattern = "\\..tif$",
                        full.names = TRUE)
r_spam <- rast(spam_files)
spam_dt <- as.data.table(as.data.frame(r_spam, xy = TRUE, na.rm = FALSE))

irrig_cols <- grep("^spam2010V2r0_global_A_.*_I$", names(spam_dt), value = TRUE)
spam_dt[, irrig_crop_area_ha := rowSums(.SD, na.rm = TRUE), .SDcols = irrig_cols]

setnames(spam_dt, c("x", "y"), c("lon", "lat"))

# Cell area on SPAM grid (ha) -----

template <- r_spam[[1]]
cell_area_ha_r <- cellSize(template, unit = "ha")

spam_dt[, cell := cellFromXY(template, cbind(lon, lat))]
spam_dt[, cell_area_ha := values(cell_area_ha_r)[cell]]

spam_dt[, irrig_extent_ha := pmin(irrig_crop_area_ha, cell_area_ha)]
spam_dt[, irrig_extent_mha := irrig_extent_ha / 1e6]

# Countries + continents -----

spam_dt <- add_country_continent(spam_dt, countries_sf, chunk_size = 200000L)

## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:

spam_dt <- spam_dt[!is.na(country)]

spam_dt[, dataset := "spam"]
spam_dt[, mha := irrig_extent_mha]

spam_dt <- spam_dt[, ..cols_to_retrieve]

# Check totals-----

# Original paper does not report a total
spam_dt[, sum(mha)]
```

```
## [1] 224.7258
```

```
# Export-----
```

```
fwrite(spam_dt, file.path(out_dir, "spam_dt.csv"))
```

2.8 GRIPC

```
# GRIPC #####
# - Irrigated area per cell given directly in ha

asc_irrig <- "./irrigated_area_datasets/GRIPC/GRIPC_irrigated_area.asc"
asc_paddy <- "./irrigated_area_datasets/GRIPC/GRIPC_paddy_area.asc"

r_irrig <- rast(asc_irrig)
r_paddy <- rast(asc_paddy)

if (!compareGeom(r_irrig, r_paddy, stopOnError = FALSE)) {
  stop("GRIPC irrigated and paddy rasters do not have matching geometry.")
}

r_irrig[is.na(r_irrig)] <- 0
r_paddy[is.na(r_paddy)] <- 0

# -----
# Country-level paddy irrigation shares reconstructed from
# Huke & Huke (1997), page 47 summary table
# share = (irrigated wet + irrigated dry) / total rice area
# -----

paddy_share_dt <- data.table(
  country = c(
    "India",
    "China",
    "Indonesia",
    "Bangladesh",
    "Thailand",
    "Vietnam",
    "Myanmar",
    "Philippines",
    "Pakistan",
    "Cambodia",
    "Nepal",
    "South Korea",
    "Sri Lanka",
    "North Korea",
    "Malaysia",
    "Lao PDR",
    "Taiwan",
    "Bhutan"
  ),
  paddy_irrig_share = c(
    (15537 + 4123) / 42516,
    (20490 + 9146) / 32125,
```



```

    (2963 + 2963) / 11015,
    (351 + 2267) / 10679,
    (274 + 665) / 9644,
    (1630 + 1630) / 6375,
    (1812 + 1386) / 6285,
    (1175 + 1029) / 3620,
    min(1, (2125 + 0) / 2124), # cap because of rounding in source
    (140 + 165) / 1899,
    (706 + 24) / 1488,
    (776 + 0) / 1103,
    (377 + 251) / 867,
    (456 + 0) / 680,
    (228 + 210) / 668,
    (33 + 11) / 611,
    min(1, (133 + 133) / 266),
    (5 + 0) / 26
  )
)

paddy_share_dt[, country:=countrycode(country, origin = "country.name",
                                       destination = "country.name")]

# Raster to table -----

irrig_dt <- as.data.table(as.data.frame(r_irrig, xy = TRUE, na.rm = FALSE))
paddy_dt <- as.data.table(as.data.frame(r_paddy, xy = TRUE, na.rm = FALSE))

setnames(irrig_dt, c("x", "y", names(irrig_dt)[3]),
          c("lon", "lat", "irrig_non_paddy_ha"))

setnames(paddy_dt, c("x", "y", names(paddy_dt)[3]),
          c("lon", "lat", "paddy_ha"))

gripc_dt <- merge(irrig_dt, paddy_dt, by = c("lon", "lat"), all = TRUE)

gripc_dt[is.na(irrig_non_paddy_ha), irrig_non_paddy_ha := 0]
gripc_dt[is.na(paddy_ha), paddy_ha := 0]

# Countries / continents -----

gripc_dt <- add_country_continent(gripc_dt, countries_sf, chunk_size = 200000L)

```

```

## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:
## Warning: Some values were not matched unambiguously: Br. Indian Ocean Ter., Fr. Polynesia, I
## Warning: Some strings were matched more than once, and therefore set to <NA> in the result:

```

```

gripc_dt <- gripc_dt[!is.na(country) & !is.na(continent)]

# Harmonize names if needed
gripc_dt[country == "Korea, Republic of", country := "South Korea"]
gripc_dt[country == "Korea, Democratic People's Republic of", country := "North Korea"]
gripc_dt[country == "Lao People's Democratic Republic", country := "Lao PDR"]
gripc_dt[country == "Taiwan, Province of China", country := "Taiwan"]
gripc_dt[country == "Burma", country := "Myanmar"]

# Join country shares-----

gripc_dt <- merge(gripc_dt, paddy_share_dt, by = "country", all.x = TRUE)

# If there is no paddy in the cell, the share is irrelevant
gripc_dt[paddy_ha == 0 & is.na(paddy_irrig_share), paddy_irrig_share := 0]

# Following Salmon et al.:
# for non-Asian countries with paddy but no country-specific value, use 0.60
gripc_dt[continent != "Asia" & paddy_ha > 0 & is.na(paddy_irrig_share),
  paddy_irrig_share := 0.60]

# Check only countries that actually have paddy area
missing_paddy_countries <- unique(
  gripc_dt[paddy_ha > 0 & is.na(paddy_irrig_share), country]
)

# Other countries, 0.60 -
gripc_dt[continent == "Asia" & paddy_ha > 0 & is.na(paddy_irrig_share),
  paddy_irrig_share := 0.60]

# Compute total irrigated area-----

gripc_dt[, irrig_paddy_ha := paddy_ha * paddy_irrig_share]
gripc_dt[, irrig_area_ha := irrig_non_paddy_ha + irrig_paddy_ha]

# Cell area raster -----

cell_ha_r <- cellSize(r_irrig, unit = "ha")

cell_dt <- as.data.table(as.data.frame(cell_ha_r, xy = TRUE, na.rm = FALSE))
setnames(cell_dt, c("x", "y", names(cell_dt)[3]), c("lon", "lat", "cell_area_ha"))

# Join to gripc_dt -----

gripc_dt <- merge(gripc_dt, cell_dt, by = c("lon", "lat"), all.x = TRUE)

gripc_dt[, `:=`(

```

```
    irrig_area_mha = irrig_area_ha / 1e6,  
    irrig_frac = irrig_area_ha / cell_area_ha  
  )]
```

```
# Final formatting -----
```

```
gripc_dt[, dataset:= "gripc"]  
gripc_dt[, mha:= irrig_area_mha]
```

```
gripc_dt <- gripc_dt[, ..cols_to_retrieve]
```

```
# Check global total -----
```

```
# Original paper reports 314 Mha  
gripc_dt[, sum(mha)]
```

```
## [1] 313.4176
```

```
# Export
```

```
fwrite(gripc_dt, file.path(out_dir, "gripc_dt.csv"))
```

2.9 MIRCA-OS

```
# MIRCA-OS (2005) #####
# - Sum irrigated crop areas per cell (Mha)

mirca_os_path <- "./irrigated_area_datasets/MIRCA_OS/mirca_os_mmgag/Maximum Monthly Growing Ar
tif_files <- list.files(path = mirca_os_path, pattern = "\\..tif$", full.names = TRUE)

ir_files <- tif_files[grepl("_ir\\.tif$", tif_files)]
ir_stack <- rast(ir_files)

mirca_os_irr_ha <- sum(ir_stack, na.rm = TRUE)
names(mirca_os_irr_ha) <- "irr_ha"

mirca_os_dt <- as.data.table(as.data.frame(mirca_os_irr_ha, xy = TRUE, na.rm = TRUE))
setnames(mirca_os_dt, c("x", "y", "irr_ha"), c("lon", "lat", "irr_ha"))

mirca_os_dt[, mha:= irr_ha / 1e6]

# Countries + continents -----

mirca_os_dt <- add_country_continent(mirca_os_dt, countries_sf, chunk_size = 200000L)

## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan, Somaliland, Taiwan
## Warning: Some values were not matched unambiguously: Kosovo, S. Sudan
mirca_os_dt <- mirca_os_dt[!is.na(country) & !is.na(continent)]

mirca_os_dt[, dataset:= "mirca_os"]
mirca_os_dt <- mirca_os_dt[, ..cols_to_retrieve]

# Check totals-----

# Original paper does not report a total
mirca_os_dt[, sum(mha)]

## [1] 271.3082

# Export-----

fwrite(mirca_os_dt, file.path(out_dir, "mirca_os_dt.csv"))
```

2.10 LUH2

```
# LUH2 (2015 snapshot) #####
# - Fraction irrigated crops × cell area to irrigated area per cell (Mha)

states_file <- "./irrigated_area_datasets/LUH2/multiple-states_input4MIPs_landState_ScenarioMIP2_2015_snapshot.nc"
mgmt_file <- "./irrigated_area_datasets/LUH2/multiple-management_input4MIPs_landState_ScenarioMIP2_2015_snapshot.nc"

nc <- nc_open(states_file)
lon <- ncvar_get(nc, "lon")
lat <- ncvar_get(nc, "lat")
time <- ncvar_get(nc, "time")
years_all <- 2015 + time
nc_close(nc)

year_to_layer <- function(year) which(years_all == year)

# Land-use fractions -----

c3ann <- rast(states_file, subds = "c3ann")
c4ann <- rast(states_file, subds = "c4ann")
c3per <- rast(states_file, subds = "c3per")
c4per <- rast(states_file, subds = "c4per")
c3nfx <- rast(states_file, subds = "c3nfx")

# Irrigation fractions -----

ir_c3ann <- rast(mgmt_file, subds = "irrig_c3ann")
ir_c4ann <- rast(mgmt_file, subds = "irrig_c4ann")
ir_c3per <- rast(mgmt_file, subds = "irrig_c3per")
ir_c4per <- rast(mgmt_file, subds = "irrig_c4per")
ir_c3nfx <- rast(mgmt_file, subds = "irrig_c3nfx")

k <- year_to_layer(2015)

c3ann_y <- c3ann[[k]]; c4ann_y <- c4ann[[k]]
c3per_y <- c3per[[k]]; c4per_y <- c4per[[k]]
c3nfx_y <- c3nfx[[k]]

ir_c3ann_y <- ir_c3ann[[k]]; ir_c4ann_y <- ir_c4ann[[k]]
ir_c3per_y <- ir_c3per[[k]]; ir_c4per_y <- ir_c4per[[k]]
ir_c3nfx_y <- ir_c3nfx[[k]]

fill <- 1.00000002004088e+20
clamp_fill <- function(r) { r[r >= fill / 10] <- NA; r }

c3ann_y <- clamp_fill(c3ann_y)
```

```

c4ann_y <- clamp_fill(c4ann_y)
c3per_y <- clamp_fill(c3per_y)
c4per_y <- clamp_fill(c4per_y)
c3nfx_y <- clamp_fill(c3nfx_y)
ir_c3ann_y <- clamp_fill(ir_c3ann_y)
ir_c4ann_y <- clamp_fill(ir_c4ann_y)
ir_c3per_y <- clamp_fill(ir_c3per_y)
ir_c4per_y <- clamp_fill(ir_c4per_y)
ir_c3nfx_y <- clamp_fill(ir_c3nfx_y)

irr_frac <- c3ann_y * ir_c3ann_y +
  c4ann_y * ir_c4ann_y +
  c3per_y * ir_c3per_y +
  c4per_y * ir_c4per_y +
  c3nfx_y * ir_c3nfx_y

cell_area_ha_luh <- cellSize(irr_frac, unit = "ha")
irr_area_ha_r <- irr_frac * cell_area_ha_luh
names(irr_area_ha_r) <- "irrig_area_ha"

luh2_dt <- as.data.table(as.data.frame(irr_area_ha_r, xy = TRUE, na.rm = FALSE))
setnames(luh2_dt, c("x", "y", "irrig_area_ha"), c("lon", "lat", "irrig_area_ha"))

# Drop NA -----

luh2_dt <- luh2_dt[!is.na(irrig_area_ha)]

# Countries + continents (chunked) -----

luh2_dt <- add_country_continent(luh2_dt, countries_sf, chunk_size = 200000L)

## Warning: Some values were not matched unambiguously: Fr. Polynesia, Fr. S. Antarctic Lands,
## Warning: Some values were not matched unambiguously: Fr. Polynesia, Fr. S. Antarctic Lands,
luh2_dt <- luh2_dt[!is.na(country) & !is.na(continent)]

luh2_dt[, mha:= irrig_area_ha / 1e6]
luh2_dt[, dataset:= "luh2"]

luh2_dt <- luh2_dt[, ..cols_to_retrieve]

# Check totals-----

# Original paper does not report a total
luh2_dt[, sum(mha)]

## [1] 267.7412

```

```
# Export-----  
fwrite(luh2_dt, file.path(out_dir, "luh2_dt.csv"))
```

3 Export

```
# EXPORT ALL DATASETS #####

# Rbind -----

irrigated_areas <- rbind(gaez_dt,
                        giam_dt,
                        gmia_dt,
                        gripc_dt,
                        luh2_dt,
                        meier_dt,
                        mirca_dt,
                        mirca_os_dt,
                        nagaraj_dt,
                        spam_dt)

# Export -----

fwrite(irrigated_areas, "./datasets/irrigated_areas/irrigated_areas.csv")
```


4 Session information

```
# SESSION INFORMATION #####
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.3.1
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] ncd4_1.24          scales_1.4.0      lubridate_1.9.4
## [4] forcats_1.0.1      stringr_1.6.0     dplyr_1.1.4
## [7] purrr_1.2.1        readr_2.2.0       tidyr_1.3.2
## [10] tibble_3.3.1       ggplot2_4.0.1.9000 tidyverse_2.0.0
## [13] readxl_1.4.5       rnatrlearn_1.1.0  sf_1.1-0
## [16] rworldmap_1.3-8    sp_2.2-1          countrycode_1.6.1
## [19] here_1.0.2         terra_1.8-93      data.table_1.18.2.1
##
## loaded via a namespace (and not attached):
## [1] dotCall64_1.2      gtable_0.3.6      spam_2.11-1       xfun_0.55
## [5] raster_3.6-32      lattice_0.22-7    tzdb_0.5.0        vctrs_0.7.1
## [9] tools_4.5.2        generics_0.1.4    proxy_0.4-29      pkgconfig_2.0.3
## [13] KernSmooth_2.23-26 RColorBrewer_1.1-3 S7_0.2.1          lifecycle_1.0.5
## [17] compiler_4.5.2     farver_2.1.2      fields_17.1       codetools_0.2-20
## [21] htmltools_0.5.9    maps_3.4.3        class_7.3-23      yaml_2.3.12
## [25] pillar_1.11.1      classInt_0.4-11   tidyselect_1.2.1  digest_0.6.39
## [29] stringi_1.8.7      rprojroot_2.1.1   fastmap_1.2.0     grid_4.5.2
## [33] cli_3.6.5          magrittr_2.0.4    e1071_1.7-17      withr_3.0.2
## [37] timechange_0.3.0    rmarkdown_2.30    otel_0.2.0        cellranger_1.1.0
## [41] hms_1.1.4          evaluate_1.0.5    knitr_1.51        viridisLite_0.4.2
## [45] rlang_1.1.7        Rcpp_1.1.1        glue_1.8.0        DBI_1.3.0
## [49] rstudioapi_0.17.1  R6_2.6.1          units_1.0-0
```

```
## Return the machine CPU -----
```

```
cat("Machine:      "); print(benchmarkme::get_cpu()$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```
## Return number of true cores -----
```

```
cat("Num cores:    "); print(parallel::detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```

```
## Return number of threads -----
```

```
cat("Num threads: "); print(parallel::detectCores(logical = FALSE))
```

```
## Num threads:
```

```
## [1] 10
```